

PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN LANJUT
SEMESTER GENAP T.A. 2025/2026
LAPORAN PROYEK AKHIR



DISUSUN OLEH:

NAMA/NIM : Muhammad Dzakwan Sundoro Adi/123250121
Azriel Aufa Risq/123250120
KELAS/PLUG : IF-F
NAMA ASISTEN : Muhammad Daud Akyaz
Bintang Shada Kawibya Putra

PROGRAM STUDI INFORMATIKA
JURUSAN INFORMATIKA
FAKULTAS TEKNIK INDUSTRI
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN"
YOGYAKARTA
2026

HALAMAN PENGESAHAN
LAPORAN PROYEK AKHIR

Disusun oleh :

Muhammad Dzakwan Sundoro Adi

123250121

Azriel Aufa Risq

123250120

Telah Diperiksa dan Disetujui oleh Asisten Praktikum Algoritma dan Pemrograman
Pada Tanggal :

Asisten Praktikum

Asisten Praktikum

Muhammad Daud Akvaz

NIM. 123240081

Bintang Shada Kawibya Putra

NIM. 123240247

KATA PENGANTAR

Puji syukur kami panjatkan kepada Tuhan Yang Maha Esa yang senantiasa mencurahkan rahmat dan hidayah-Nya sehingga kami dapat menyelesaikan Praktikum Algoritma Lanjut serta laporan akhir Algoritma Lanjut. Adapun laporan ini berisi tentang proyek akhir semester dari hasil pembelajaran selama praktikum berlangsung.

Tidak lupa ucapan terima kasih kepada asisten dosen yang selalu membimbing dan mengajari kami dalam melaksanakan praktikum dan dalam menyusun laporan ini. Laporan ini masih sangat jauh dari kesempurnaan, oleh karena itu kritik serta saran yang membangun kami harapkan untuk menyempurnakan laporan akhir ini.

Atas perhatian dari semua pihak yang membantu penulisan ini, kami ucapkan terima kasih. Semoga laporan ini dapat dipergunakan seperlunya.

Yogyakarta, 09 Juni 2026

Penyusun I



Muhammad Dzakwan Sundoro Adi

NIM. 123250121

Penyusun II



Azriel Aufa Risq

NIM. 123250120

DAFTAR ISI

HALAMAN PENGESAHAN	i
KATA PENGANTAR	ii
DAFTAR ISI	iii
BAB I	1
BAB II.....	3
3.1. Implementasi Materi.....	4
3.2. Penjelasan Fitur	9
3.3. Screenshot Program.....	16
3.4. Source Code Lengkap.....	24
BAB IV	51
4.1. Jadwal Pengerjaan	51
4.2. Pembagian Tugas.....	51
BAB V	53
5.1 Kesimpulan.....	53
5.2 Saran	53
DAFTAR PUSTAKA	54

BAB I

Manajemen Toko Buku

1.1 Deskripsi Program

Program ini adalah sebuah aplikasi *Command Line Interface* (CLI) berbasis C++ dengan nama "Sistem Manajemen Toko Buku Sinar Abadi". Konsep utama aplikasi ini adalah Sistem Manajemen Data Berbasis *File*. Program ini membaca, menyimpan, dan memperbarui data secara *persisten* melalui operasi *file (txt file)*. Seluruh alur data, termasuk inventaris buku, riwayat transaksi, dan keanggotaan, dikelola dalam memori saat program berjalan, lalu disinkronkan kembali ke dalam *file* secara otomatis setiap kali terjadi perubahan.

1.2 Fitur-Fitur Program

1. Autentikasi & Hak Akses: Sistem *login* yang memisahkan jalur akses untuk Admin dan Pelanggan/*Member*. Terdapat batasan 3 kali percobaan *login* sebelum akses ditolak.
2. Panel Admin:
 - a) Manajemen Inventaris: Operasi untuk menambahkan buku ke rak, menambah stok, dan menghapus buku.
 - b) Manajemen Pengguna: Admin dapat melihat daftar *Member*, menonaktifkannya, atau menghapusnya secara permanen.
 - c) Riwayat Transaksi: Pencatatan seluruh transaksi pembelian buku secara detail.
3. Panel Pelanggan:
 - a) Katalog dan Pencarian: Penelusuran buku berdasarkan kategori (Fiksi/Non-Fiksi) maupun pencarian spesifik berbasis judul.
 - b) Buku Terlaris: Menampilkan daftar buku *best seller* berdasarkan angka penjualan tertinggi.
 - c) Sistem Transaksi: Pembelian buku berdasarkan koordinat rak dan *slot*, dilengkapi sistem diskon otomatis sebesar 5% khusus bagi pelanggan dengan status *Membership*.

1.3 Topik-Topik Terkait

Program ini mengimplementasikan konsep-konsep Algoritma dan Pemrograman Lanjut, yaitu:

1. *Array*: Penggunaan *array* 1 dimensi untuk menyimpan sekumpulan data seperti daftar buku dan *member*, serta teknik pemetaan indeks untuk menyimulasikan *array* 2 dimensi (sistem rak dan *slot*).
2. *Struct*: Tipe data bentukan untuk merepresentasikan entitas dengan berbagai atribut, seperti buku, pelanggan, dan admin.
3. *Rekursif*:
4. *Searching*: Implementasi *Linear Search* untuk mencari data spesifik (misal: mencari *username* saat *login* atau mencari judul buku).
5. *Sorting*: Pengurutan data buku *Best Seller* dari penjualan tertinggi ke terendah menggunakan algoritma *Bubble Sort*.
6. *Pointer*: Pemanfaatan *array of pointers* saat melakukan pengurutan data (*sorting*) untuk menghemat memori tanpa perlu memindahkan isi data *struct* secara keseluruhan.
7. *File I/O*: Manajemen penyimpanan data *persisten* menggunakan *library fstream* (*ifstream* untuk membaca dan *ofstream* untuk menulis data).

BAB II

TUJUAN PROYEK AKHIR

Tujuan dari proyek akhir ini adalah untuk mengimplementasikan dan menguji pemahaman mahasiswa terhadap materi-materi Algoritma dan Pemrograman Lanjut.

Aplikasi ini dibuat dengan tujuan untuk mengotomatisasi proses bisnis di dalam sebuah toko buku. Aplikasi ini dirancang agar dapat memfasilitasi peran pengelola dalam mengorganisasi manajemen inventaris dan memantau riwayat penjualan, sekaligus memfasilitasi peran pembeli (Pelanggan) untuk menelusuri katalog, melakukan pencarian buku spesifik, hingga melakukan transaksi pembelian buku dengan sistem pemotongan harga otomatis bagi *membership* terdaftar.

BAB III

PEMBAHASAN

3.1. Implementasi Materi

3.1.1 *Struct*

Struct digunakan sebagai wadah pembentuk tipe data bentukan yang mengelompokkan berbagai atribut (seperti judul, penulis, harga, stok, dan status ketersediaan) ke dalam satu objek buku yang utuh.

```
struct buku {  
    int rak;  
    int slot;  
    string namaBuku;  
    string namaPenulis;  
    int tahunTerbit;  
    string kategori;  
    int harga;  
    int jumlahBest;  
    int jumlahStok;  
    bool isTersedia;  
};  
  
struct pelanggan {  
    string namaPelanggan;  
    string tanggal;  
    string namaBukuDibeli;  
    int jumlahBeli;  
};  
  
struct member {  
    string username;  
    string password;  
    bool isTersedia;  
};  
  
struct admin {  
    string username;
```



```

    string password;
};

```

3.1.2 Array

Array digunakan untuk wadah memori statis utama yang menampung sekumpulan data entitas selama program berjalan. Pada proyek ini, kami memetakan *array* satu dimensi agar berfungsi layaknya struktur dua dimensi (pada *array rakBuku*) untuk mensimulasikan tata letak fisik rak dan *slot* buku.

```

const int MAX_RAK = 5, MAX_SLOT = 20;
const int MAX_MEMBER = 100;
const int MAX_ADMIN = 100;
buku rakBuku[MAX_RAK * MAX_SLOT];
member membership[MAX_MEMBER];
admin staff[MAX_ADMIN];

```

3.1.3 Rekursif

Rekursif digunakan untuk menangani operasi yang membutuhkan pemanggilan diri sendiri secara berulang, seperti pada fungsi *looping* validasi *login*.

```

    if (maxAttempt == 3) {
        (type == "A" ? loadDatabaseAdmin() :
loadDatabaseMember());
    }
    if (maxAttempt <= 0) {
        cout << "Login Gagal! Kesempatan Anda telah habis.\n";
        aksiSelesai();
        return false;
    }
    .....

    if (type == "A") {
        for (int i = 0; i < jumlah_admin; i++) {

```

```

        if (u == staff[i].username && p ==
staff[i].password) {
            .....
            return true;
        }
    }
}
else {
    for (int i = 0; i < jumlah_member; i++) {
        if (u == membership[i].username && p ==
membership[i].password) {
            if (membership[i].isTersedia) {
                .....
                return true;
            } else {
                maxAttempt--;
                .....
                return prosesLogin(type, maxAttempt);
            }
        }
    }
}
maxAttempt--;
.....
return prosesLogin(type, maxAttempt);
}

```

3.1.4 Searching

searching digunakan untuk memudahkan pengguna dalam melakukan pencarian informasi. Metode yang digunakan adalah *Linear Search*, di mana program memeriksa setiap elemen *array* satu per satu.

```

bool f = false;
for (int i = 0; i < MAX_RAK * MAX_SLOT; i++) {
    .....
}

```

```

        f = true;
    }
}
if (!f) cout << "Buku \"" << keyword << "\" tidak
ditemukan.\n";
}

```

3.1.5 Sorting

sorting digunakan untuk menampilkan data yang terorganisir (mengurut), khususnya pada fitur *Best Seller*.

```

for (int i = 0; i < jml - 1; i++)
    for (int j = 0; j < jml - i - 1; j++)
        if (daftarPointer[j]->jumlahBest <
daftarPointer[j+1]->jumlahBest) {
            buku* temp = daftarPointer[j];
            daftarPointer[j] = daftarPointer[j+1];
            daftarPointer[j+1] = temp;
        }

```

3.1.1 Pointer

Pointer digunakan untuk mengoptimalkan kinerja sistem saat melakukan proses pengurutan (*sorting*) atau manipulasi data yang kompleks.

```

buku* daftarPointer[MAX_RAK * MAX_SLOT]; // pada fungsi sort
buku
buku* ptrBuku = &rakBuku[i]; // pada fungsi cari judul buku

```

3.1.6 File I/O

File I/O dengan *library fstream* digunakan untuk menerapkan alur kerja di mana data dari *file (txt file)* dimuat secara utuh ke dalam *array* memori sesaat setelah program dijalankan menggunakan fungsi *ifstream*. Kemudian, setiap kali terjadi modifikasi data (seperti transaksi pembelian, penambahan stok, atau pendaftaran *member* baru) fungsi *ofstream* akan dipanggil untuk memperbarui isi

file, sehingga rekam jejak data selalu sinkron antara memori dan penyimpanan lokal.

```
void loadDatabaseBuku() { // Output file
    ifstream file(File_Buku);
    if (!file.is_open()) return;
    string line;
    while (getline(file, line)) {
        if (line.empty()) continue;
        stringstream ss(line);
        string temp;
        buku buku_temp;

        getline(ss, temp, ' '); if (!temp.empty())
buku_temp.rak = stoi(temp);
        getline(ss, temp, ' '); if (!temp.empty())
buku_temp.slot = stoi(temp);
        getline(ss, buku_temp.namaBuku, ' ');
        getline(ss, buku_temp.namaPenulis, ' ');
        getline(ss, temp, ' '); if (!temp.empty())
buku_temp.tahunTerbit = stoi(temp);
        getline(ss, buku_temp.kategori, ' ');
        getline(ss, temp, ' '); if (!temp.empty())
buku_temp.harga = stoi(temp);
        getline(ss, temp, ' '); if (!temp.empty())
buku_temp.jumlahBest = stoi(temp);
        getline(ss, temp, ' '); if (!temp.empty())
buku_temp.jumlahStok = stoi(temp);
        getline(ss, temp); if (!temp.empty())
buku_temp.isTersedia = (temp == "1" ? true : false);

        buku_temp.namaBuku =
replaceUnderscoretoSpace(buku_temp.namaBuku);
        buku_temp.namaPenulis =
replaceUnderscoretoSpace(buku_temp.namaPenulis);
        buku_temp.kategori =
```

```

replaceUnderscoretoSpace(buku_temp.kategori);

        rakBuku[((buku_temp.rak - 1) * MAX_SLOT) +
(buku_temp.slot - 1)] = buku_temp;
    }
    file.close();
}

void saveDatabaseBuku() { // Input file
    ofstream file(File_Buku, ios::trunc);
    if (!file.is_open()) return;
    for (int i = 0; i < MAX_RAK * MAX_SLOT; i++) {
        int nomorRak = (i / MAX_SLOT) + 1;
        int nomorSlot = (i % MAX_SLOT) + 1;
        if (rakBuku[i].isTersedia) {
            file << nomorRak << " " << nomorSlot << " " <<
replaceSpacetoUnderscore(rakBuku[i].namaBuku)
                << " " <<
replaceSpacetoUnderscore(rakBuku[i].namaPenulis)
                << " " << rakBuku[i].tahunTerbit << " " <<
replaceSpacetoUnderscore(rakBuku[i].kategori)
                << " " << rakBuku[i].harga << " " <<
rakBuku[i].jumlahBest << " " << rakBuku[i].jumlahStok
                << " " << rakBuku[i].isTersedia << "\n";
        }
    }
    file.close();
}

```

3.2. Penjelasan Fitur

3.2.1 Menambahkan Buku (Admin)

Fitur ini memungkinkan Admin untuk mendaftarkan buku baru tanpa perlu mencari lokasi rak secara manual. Program melakukan iterasi (perulangan) pada keseluruhan kapasitas *array* untuk mencari indeks pertama yang kosong (ditandai dengan *!isTersedia*). Setelah indeks ditemukan, program mengonversinya

menjadi nomor Rak dan *Slot*, lalu meminta *input* data buku dari Admin dan menyimpannya.

```
int indeks = -1;
for (int i = 0; i < MAX_RAK * MAX_SLOT; i++) {
    if (!rakBuku[i].isTersedia) {
        indeks = i;
        break;
    }
}
.....
rakBuku[indeks].jumlahBest = 0;
rakBuku[indeks].isTersedia = true;
saveDatabaseBuku();
```

3.2.2 Menambah Stok Buku (Admin)

Admin memasukkan koordinat Rak dan *Slot* yang spesifik. Program mengonversi *input* tersebut menjadi indeks *array linear*, menampilkan data buku lama untuk konfirmasi, lalu menambahkan nilai stok baru (*tamAngka*) ke variabel *jumlahStok* yang sudah ada di memori, dan menyimpannya kembali ke basis data.

```
int indeks = ((rak - 1) * MAX_SLOT) + slot - 1;
cout << "Nama Buku: " << rakBuku[indeks].namaBuku << "\n";
cout << "Stok lama: " << rakBuku[indeks].jumlahStok << "\n";
.....
rakBuku[indeks].jumlahStok += tamAngka; // Menambahkan stok
baru
saveDatabaseBuku();
```

3.2.3 Menghapus Buku (Admin)

Penghapusan buku menggunakan pergeseran elemen. Daripada sekadar mengosongkan indeks dan membiarkan adanya "lubang" di tengah-tengah rak, program menggeser seluruh elemen buku yang ada di sebelah kanan indeks sasaran ke arah kiri, sehingga urutan rak buku tetap rapat.

```

void hapusBuku(int ind) { dihapus
    for (int j = ind; j < (MAX_RAK * MAX_SLOT) - 1; j++) {
        if (rakBuku[j].isTersedia) {
            rakBuku[j] = rakBuku[j + 1];
        }
    }
    saveDatabaseBuku();
    cout << "sudah dihapus.\n";
}

```

3.2.4 Melihat Daftar Member (Admin)

Program melakukan iterasi *Linear Search* dari indeks ke-0 hingga batas total *jumlah_member* saat ini. Fungsi ini mencetak atribut *username* dan status ketersediaannya dengan menggunakan *ternary* operator agar lebih ringkas.

```

cout << "=== DAFTAR MEMBER ===\n";
if (jumlah_member == 0) cout << "[ Belum ada data member ]\n";
for (int i = 0; i < jumlah_member; i++) {
    cout << "Username: " << membership[i].username << "\n"
        << "Status: " << (membership[i].isTersedia ? "Aktif" :
"Nonaktif") << "\n\n";
}

```

3.2.5 Menonaktifkan Member (Admin)

Program mencari *username* yang di-input-kan oleh Admin. Jika ditemukan, program tidak menghapus datanya, melainkan hanya mengubah parameter *isTersedia* menjadi *false*. Hal ini membuat *member* tersebut tidak akan lolos validasi saat mencoba *login* ke depannya.

```

for (int i = 0; i < jumlah_member; i++) {
    if (membership[i].username == target) {
        if (!membership[i].isTersedia) {
            cout << "Member memang sudah dalam keadaan
nonaktif.\n";
        } else {

```

```

        membership[i].isTersedia = false; // Soft delete
        saveAllMembers();
    }
    break;
}
}

```

3.2.6 Menghapus Member (Admin)

Serupa dengan logika penghapusan buku, jika Admin memilih penghapusan permanen, program akan menggeser elemen *array membership* untuk menimpa akun yang dihapus dan menurunkan nilai variabel penghitung *jumlah_member*.

```

for (int i = 0; i < jumlah_member; i++) {
    if (membership[i].username == target) {
        for (int j = i; j < jumlah_member - 1; j++) {
            membership[j] = membership[j + 1];
        }
        jumlah_member--; // Mengurangi total member
        saveAllMembers();
        break;
    }
}
}

```

3.2.7 Melihat Riwayat Transaksi (Admin)

Program membuka *file DatabaseRiwayat.txt* menggunakan pustaka *ifstream*. Dengan fungsi *getline()*, program membaca *file* baris demi baris, memisahkan atribut menggunakan *stringstream*, menyimpannya sementara ke *struct* pelanggan, dan mencetaknya ke layar.

```

ifstream file(File_Riwayat);
string line;
while (getline(file, line)) {
    if (line.empty()) continue;
    stringstream ss(line);
    pelanggan pelanggan_temp;

```



```

        getline(ss, pelanggan_temp.namaPelanggan, ' ');
        .....
        cout << "Pelanggan: " <<
        replaceUnderscoretoSpace(pelanggan_temp.namaPelanggan) << "\n"
            << "Buku: " <<
        replaceUnderscoretoSpace(pelanggan_temp.namaBukuDibeli) << "\n"
            << "Jml: " << pelanggan_temp.jumlahBeli << " pcs\n---
        -----\n";
    }
    file.close();

```

3.2.8 Mencari Buku Berdasarkan Kategori atau Judul (*User/Member*)

Pencarian diterapkan dengan membandingkan *input string* pengguna terhadap *array* buku. Khusus untuk pencarian kategori, program dilengkapi dengan dua variabel penampung (*Upper Case* dan *Lower Case*) agar proses pencarian menjadi tidak peka huruf besar/kecil.

```

void kategoribuku() {
    .....
    string katTarget = (pilAngka == 1) ? "Fiksi" : "Non-Fiksi";
    string katTargetLower = (pilAngka == 1) ? "fiksi" : "non-
    fiksi";

    for (int i = 0; i < MAX_RAK * MAX_SLOT; i++) {
        if (rakBuku[i].isTersedia && (rakBuku[i].kategori ==
        katTarget || rakBuku[i].kategori == katTargetLower)) {
            .....
        }
    }
}

void cariBukuJudul() {
    string keyword;
    cout << "=== CARI JUDUL BUKU ===\n"
        << "Masukkan Judul: ";
    getline(cin, keyword);

```

```

bool f = false;
for (int i = 0; i < MAX_RAK * MAX_SLOT; i++) {
    buku* ptrBuku = &rakBuku[i];
    int nomorRak = (i / MAX_SLOT) + 1;
    int nomorSlot = (i % MAX_SLOT) + 1;
    if ((ptrBuku->isTersedia) && ptrBuku->namaBuku ==
keyword) { .....
        f = true;
    }
}
if (!f) cout << "Buku \"" << keyword << "\" tidak
ditemukan.\n";}

```

3.2.9 Menampilkan Buku Dengan Penjualan Tertinggi (*User/Member*)

Fitur ini mengurutkan buku dari penjualan terbanyak menggunakan *Bubble Sort*. Untuk menghindari lambatnya eksekusi karena memindahkan *struct* buku berukuran besar di memori, algoritma *sorting* dioptimalkan hanya dengan menukar alamat *pointer* referensinya saja (buku* daftarPointer).

```

for (int i = 0; i < jml - 1; i++) {
    for (int j = 0; j < jml - i - 1; j++) {
        if (daftarPointer[j]->jumlahBest < daftarPointer[j+1]-
>jumlahBest) {
            // Proses Swap Pointer
            buku* temp = daftarPointer[j];
            daftarPointer[j] = daftarPointer[j+1];
            daftarPointer[j+1] = temp;
        }
    }
}

```

3.2.10 Pembelian Buku Diskon 5% Untuk Member (*User/Member*)

Saat *login* berhasil sebagai Member, program mengirim variabel diskon = 0.05 ke parameter fungsi pembelian. Selama proses transaksi, program memotong

stok buku, mengalkulasi harga final, mengubah parameter ketersediaan jika stok habis, dan merekam data ke *file* histori.

```
cout << "Harga (perbuku): " << rakBuku[indeks].harga -
(rakBuku[indeks].harga * diskon) << "\n";

// Mengurangi stok buku sesuai jumlah beli
rakBuku[indeks].jumlahStok -= jmlBeli;

if (rakBuku[indeks].jumlahStok == 0){
    rakBuku[indeks].isTersedia = false;
}
rakBuku[indeks].jumlahBest += jmlBeli; // Menambah poin best
seller

p.namaBukuDibeli = rakBuku[indeks].namaBuku;
p.jumlahBeli = jmlBeli;
saveDatabaseBuku();
saveTransaksi(p);
```

3.2.11 Melihat Daftar Buku (*User*/Member dan Admin)

Menampilkan seluruh inventaris buku. Program melakukan iterasi penuh pada memori rak buku dan mengonversi kembali indeks (linear/1 dimensi) menjadi koordinat visual (Rak dan *Slot*/2 dimensi) ke layar dengan memanfaatkan operator aritmatika / (pembagian) dan % (*modulo*).

```
for (int i = 0; i < MAX_RAK * MAX_SLOT; i++) { Slot
    int nomorRak = (i / MAX_SLOT) + 1;
    int nomorSlot = (i % MAX_SLOT) + 1;

    if (rakBuku[i].isTersedia) {
        cout << " -> [Rak " << nomorRak << ", Slot " <<
nomorSlot << "]" << "\n"
        << "Nama Buku: " << rakBuku[i].namaBuku << "\n"
        << "Stok: " << rakBuku[i].jumlahStok << "
pcs\n\n";
```

```
}  
}
```

3.3. Screenshot Program

```
=== MENU UTAMA ===  
1. Pelanggan / Member  
2. Petugas Admin  
3. Keluar  
Pilih:
```

Menu Utama

```
=== LOGIN MEMBER ===  
Username: budi ganteng  
Password: rahasia123  
Login Berhasil sebagai Member!  
Anda mendapatkan diskon 5%  
-----  
Press any key to continue . . . |
```

Login Member

```
=== TOKO BUKU SINAR ABADI ===  
1. Daftar Semua Buku  
2. Cari via Kategori  
3. Cari Judul  
4. Best Seller  
5. Beli Buku  
6. Keluar  
Pilih: |
```

Menu User/Member

```
=== KATEGORI BUKU ===  
1. Fiksi  
2. Non-Fiksi  
3. Kembali  
Pilih: |
```

Menu *User/Member 2* (Cari Kategori)

```
=== Fiksi ===  
-> [Rak 1][Slot 1]  
Nama Buku      : Laskar Pelangi  
Harga (Diskon): 85000  
Stok Tersedia: 5 pcs  
-----  
-> [Rak 1][Slot 2]  
Nama Buku      : Bumi Manusia  
Harga (Diskon): 99000  
Stok Tersedia: 3 pcs  
-----  
-> [Rak 1][Slot 3]  
Nama Buku      : Negeri 5 Menara  
Harga (Diskon): 75000  
Stok Tersedia: 7 pcs  
-----  
-> [Rak 1][Slot 4]  
Nama Buku      : Pulang  
Harga (Diskon): 88000  
Stok Tersedia: 4 pcs  
-----
```

Pilihan Fiksi

```

=== Non-Fiksi ===
-> [Rak 1][Slot 5]
Nama Buku      : Filosofi Teras
Harga (Diskon): 78000
Stok Tersedia: 10 pcs
-----
-> [Rak 1][Slot 6]
Nama Buku      : Sebuah Seni untuk Memahami Kekasih
Harga (Diskon): 68000
Stok Tersedia: 15 pcs
-----
-> [Rak 1][Slot 7]
Nama Buku      : Sapiens
Harga (Diskon): 125000
Stok Tersedia: 2 pcs
-----

```

Pilihan Non-Fiksi

```

=== CARI JUDUL BUKU ===
Masukkan Judul: Laskar Pelangi
-> [Rak 1, Slot 1]
Nama Buku: Laskar Pelangi
Penulis: Andrea Hirata
Tahun Terbit: 2005
Kategori: Fiksi
Harga: 85000
Stok: 5 pcs
-----
Press any key to continue . . . |

```

Menu *User/Member* 3 (Cari Judul)

```
=== DAFTAR BUKU BEST SELLER ===  
Judul: Laskar Pelangi  
Penulis: Andrea Hirata  
Tahun Terbit: 2005  
Kategori: Fiksi  
Harga: 85000  
Terjual: 50 pcs  
Sisa: 5  
-----  
Judul: Bumi Manusia  
Penulis: Pramoedya Ananta Toer  
Tahun Terbit: 1980  
Kategori: Fiksi  
Harga: 99000  
Terjual: 30 pcs  
Sisa: 3  
-----
```

Menu *User/Member* 4 (Best Seller)

```
=== BELI BUKU ===  
Masukkan No Rak & Slot (misal: 1 2): 1 1  
  
Buku Terpilih:  
Nama Buku      : Laskar Pelangi  
Harga (perbuku): 80750  
Stok Tersedia  : 5 pcs  
Jumlah Beli: 1  
Nama Anda: Dzakwan  
Tanggal (DD-MM-YYYY): 07-06-2026  
  
Pembelian Berhasil!  
-----  
Press any key to continue . . . |
```

Menu *User/Member* 5 (Beli Buku)

```
=== LOGIN ADMIN ===  
Username: adminaufa  
Password: adminaufa123  
Login Berhasil sebagai Admin!  
-----  
Press any key to continue . . . |
```

Login Admin

```
=== MENU ADMIN ===  
1. Daftar Semua Buku  
2. Tambah Buku  
3. Tambah Stok  
4. Hapus Buku  
5. Riwayat Transaksi  
6. Kelola Member  
7. Keluar  
Pilih: |
```

Menu Admin

```
=== TAMBAH BUKU (Otomatis) ===  
Lokasi Buku: Rak 1, Slot 17  
-----  
Judul: Yanto Sang Pencari Rupiah  
Penulis: Kursu Rahardjo  
Tahun Terbit: 2007  
Kategori (Fiksi/Non-Fiksi): Non-Fiksi  
Harga: 45000  
Stok: 400  
  
Buku berhasil didaftarkan!  
-----  
Press any key to continue . . . |
```

Menu Admin 1 (Tambah Buku)


```

Masukkan No Rak & Slot (misal 1 2): 1 17
Nama Buku: Yanto Sang Pencari Rupiah
Stok lama: 400
Tambahkan Stok: 20
Stok diperbarui!
-----
Press any key to continue . . . |

```

Menu Admin 2 (Tambah Stok)

```

Masukkan No Rak & Slot (misal 1 2): 1 17
Nama Buku: Yanto Sang Pencari Rupiah
sudah dihapus.
-----
Press any key to continue . . . |

```

Menu Admin 3 (Hapus Buku)

```

=== RIWAYAT PEMBELIAN BUKU ===
Pelanggan: Budi Hermawan
Tgl: 02-06-2026
Buku: Laskar Pelangi
Jml: 2 pcs
-----
Pelanggan: Siti Rahma
Tgl: 02-06-2026
Buku: Filosofi Teras
Jml: 1 pcs
-----
Pelanggan: Ahmad Subarjo
Tgl: 03-06-2026
Buku: Atomic Habits
Jml: 3 pcs
-----
Pelanggan: Dewi Lestari
Tgl: 04-06-2026
Buku: Perahu Kertas
Jml: 1 pcs
-----

```

Menu Admin 4 (Riwayat Transaksi)

```

=== KELOLA MEMBERSHIP ===
1. Daftar Semua Member
2. Nonaktifkan Member
3. Hapus Member Permanen
4. Kembali ke Menu Admin
Pilih: |

```

Menu Admin 5 (Kelola Member)

```

=== DAFTAR MEMBER ===
Username: budi ganteng
Status: Aktif

Username: siti imut
Status: Aktif

Username: ahmad subarjo
Status: Aktif

Username: dewi lestari
Status: Aktif

Username: rian hidayat
Status: Aktif

Username: enggar
Status: Aktif

Username: dzakwan
Status: Aktif

```

Daftar Member (Kelola Member)

```

=== NONAKTIFKAN MEMBER ===
Masukkan Username Member: budi ganteng
Member "budi ganteng" berhasil dinonaktifkan!
-----
Press any key to continue . . . |

```

Menonaktifkan Member (Kelola Member)

```
=== HAPUS MEMBER PERMANEN ===  
Masukkan Username Member yang akan dihapus: budi ganteng  
Member "budi ganteng" berhasil dihapus secara permanen.  
-----  
Press any key to continue . . . |
```

Menghapus Member (Kelola *Member*)

```
=====membuat member=====  
silahkan masukkan  
Username: budi ganteng  
Password: rahasia123  
berhasil kamu langsung masuk sebagai membership  
Anda mendapatkan diskon 5%  
-----  
Press any key to continue . . . |
```

Membuat *Member* Baru (*User*)

```

=== DAFTAR BUKU ===

-> [Rak 1, Slot 1]
Nama Buku: Laskar Pelangi
Penulis: Andrea Hirata
Tahun Terbit: 2005
Kategori: Fiksi
Harga: 85000
Stok: 5 pcs

-> [Rak 1, Slot 2]
Nama Buku: Bumi Manusia
Penulis: Pramoedya Ananta Toer
Tahun Terbit: 1980
Kategori: Fiksi
Harga: 99000
Stok: 3 pcs

-> [Rak 1, Slot 3]
Nama Buku: Negeri 5 Menara
Penulis: A. Fuadi
Tahun Terbit: 2009
Kategori: Fiksi
Harga: 75000
Stok: 7 pcs

```

Daftar Buku (Menu Admin dan Menu *User/Member*)

3.4. Source Code Lengkap

```

#include <iostream>
#include <string>
#include <fstream>
#include <sstream>
using namespace std;

struct buku {
    int rak;
    int slot;
    string namaBuku;
    string namaPenulis;
    int tahunTerbit;

```

```
    string kategori;
    int harga;
    int jumlahBest;
    int jumlahStok;
    bool isTersedia;
};

struct pelanggan {
    string namaPelanggan;
    string tanggal;
    string namaBukuDibeli;
    int jumlahBeli;
};

struct member {
    string username;
    string password;
    bool isTersedia;
};

struct admin {
    string username;
    string password;
};

const int MAX_RAK = 5, MAX_SLOT = 20;
const int MAX_MEMBER = 100;
const int MAX_ADMIN = 100;
const string File_Buku = ("data/DatabaseBuku.txt");
const string File_Riwayat = ("data/DatabaseRiwayat.txt");
const string File_Member = ("data/DatabaseMember.txt");
const string File_Admin = ("data/DatabaseAdmin.txt");
buku rakBuku[MAX_RAK * MAX_SLOT];
member membership[MAX_MEMBER];
admin staff[MAX_ADMIN];
```

```
int jumlah_member = 0;
int jumlah_admin = 0;

string replaceChar(string text, char source, char target){
    string newtext = text;
    for(int i = 0; i < text.length(); i ++){
        if(text[i] == source)
            newtext[i] = target;
    }
    return newtext;
}

string replaceSpacetoUnderscore(string text){
    return replaceChar(text, ' ', '_');
}

string replaceUnderscoretoSpace(string text){
    return replaceChar(text, '_', ' ');
}

bool cek_input(string input, int& hasil) {
    if (input.empty()) return false;

    stringstream ss(input);
    int angka;
    char sisa;
    if (ss >> angka) {
        if (ss >> sisa) {
            return false;
        }
        hasil = angka;
        return true;
    }
    return false;
}
```

```

void aksiSelesai() {
    cout << "-----\n";
    system("pause"); system("cls");
}

void loadDatabaseBuku() {
    ifstream file(File_Buku);
    if (!file.is_open()) return;
    string line;
    while (getline(file, line)) {
        if (line.empty()) continue;
        stringstream ss(line);
        string temp;
        buku buku_temp;

        getline(ss, temp, ' '); if (!temp.empty()) buku_temp.rak =
stoi(temp);
        getline(ss, temp, ' '); if (!temp.empty()) buku_temp.slot =
stoi(temp);
        getline(ss, buku_temp.namaBuku, ' ');
        getline(ss, buku_temp.namaPenulis, ' ');
        getline(ss, temp, ' '); if (!temp.empty())
buku_temp.tahunTerbit = stoi(temp);
        getline(ss, buku_temp.kategori, ' ');
        getline(ss, temp, ' '); if (!temp.empty()) buku_temp.harga =
stoi(temp);
        getline(ss, temp, ' '); if (!temp.empty())
buku_temp.jumlahBest = stoi(temp);
        getline(ss, temp, ' '); if (!temp.empty())
buku_temp.jumlahStok = stoi(temp);
        getline(ss, temp); if (!temp.empty()) buku_temp.isTersedia =
(temp == "1" ? true : false);

        buku_temp.namaBuku =

```

```

replaceUnderscoretoSpace(buku_temp.namaBuku);
        buku_temp.namaPenulis =
replaceUnderscoretoSpace(buku_temp.namaPenulis);
        buku_temp.kategori =
replaceUnderscoretoSpace(buku_temp.kategori);

        rakBuku[((buku_temp.rak - 1) * MAX_SLOT) + (buku_temp.slot -
1)] = buku_temp;
    }
    file.close();
}

void saveDatabaseBuku() {
    ofstream file(File_Buku, ios::trunc);
    if (!file.is_open()) return;
    for (int i = 0; i < MAX_RAK * MAX_SLOT; i++) {
        int nomorRak = (i / MAX_SLOT) + 1;
        int nomorSlot = (i % MAX_SLOT) + 1;
        if (rakBuku[i].isTersedia) {
            file << nomorRak<< " " << nomorSlot << " " <<
replaceSpacetoUnderscore(rakBuku[i].namaBuku)
                << " " <<
replaceSpacetoUnderscore(rakBuku[i].namaPenulis)
                << " " << rakBuku[i].tahunTerbit << " " <<
replaceSpacetoUnderscore(rakBuku[i].kategori)
                << " " << rakBuku[i].harga << " "<<
rakBuku[i].jumlahBest << " " << rakBuku[i].jumlahStok
                << " " << rakBuku[i].isTersedia << "\n";
        }
    }
    file.close();
}

void lihatRiwayatPembelian() {
    ifstream file(File_Riwayat);

```



```

cout << "=== RIWAYAT PEMBELIAN BUKU ===\n";
if (!file.is_open()) {
    cout << "[ Belum ada data transaksi ]\n";
    aksiSelesai(); return;
}
string line; bool ada = false;
while (getline(file, line)) {
    if (line.empty()) continue;
    stringstream ss(line);
    string temp;
    pelanggan pelanggan_temp;

    getline(ss, pelanggan_temp.namaPelanggan, ' ');
    getline(ss, pelanggan_temp.tanggal, ' ');
    getline(ss, pelanggan_temp.namaBukuDibeli, ' ');
    getline(ss, temp); if (!temp.empty())
    pelanggan_temp.jumlahBeli = stoi(temp);

    pelanggan_temp.namaPelanggan =
    replaceUnderscoretoSpace(pelanggan_temp.namaPelanggan);
    pelanggan_temp.namaBukuDibeli =
    replaceUnderscoretoSpace(pelanggan_temp.namaBukuDibeli);
    cout << "Pelanggan: " << pelanggan_temp.namaPelanggan <<
"\n"
        << "Tgl: " << pelanggan_temp.tanggal << "\n"
        << "Buku: " << pelanggan_temp.namaBukuDibeli << "\n"
        << "Jml: " << pelanggan_temp.jumlahBeli << " pcs\n-----
-----\n";

    ada = true;
}
if (!ada) cout << "[ Belum ada data transaksi ]\n";
file.close();
}

void saveTransaksi(pelanggan p) {

```

```

    ofstream file(File_Riwayat, ios::app);
    if (!file.is_open()) return;

    file << replaceSpacetoUnderscore(p.namaPelanggan) << " " <<
    replaceSpacetoUnderscore(p.tanggal) << " "
        << replaceSpacetoUnderscore(p.namaBukuDibeli) << " " <<
    p.jumlahBeli << "\n";
    file.close();
}

void loadDatabaseMember() {
    ifstream file(File_Member);
    if (!file.is_open()) return;
    string line;
    jumlah_member = 0;
    while (getline(file, line)) {
        if (line.empty()) continue;
        stringstream ss(line);
        string temp;
        member member_temp;

        getline(ss, member_temp.username, ' ');
        getline(ss, member_temp.password, ' ');
        getline(ss, temp); if (!temp.empty()) member_temp.isTersedia
= (temp == "1" ? true : false);

        member_temp.username =
        replaceUnderscoretoSpace(member_temp.username);
        member_temp.password =
        replaceUnderscoretoSpace(member_temp.password);
        membership[jumlah_member] = member_temp;
        jumlah_member++;
    }
    file.close();
}

```

```

void saveDatabaseMember(member m) {
    ofstream file(File_Member, ios::app);
    if (!file.is_open()) return;
    file << replaceSpacetoUnderscore(m.username) << " " <<
replaceSpacetoUnderscore(m.password)
    << " " << m.isTersedia << "\n";
    file.close();

    cout << "berhasil kamu langsung masuk sebagai membership\n";
}

void saveAllMembers() {
    ofstream file(File_Member, ios::trunc);
    if (!file.is_open()) return;

    for (int i = 0; i < jumlah_member; i++) {
        file << replaceSpacetoUnderscore(membership[i].username) <<
" "
        << replaceSpacetoUnderscore(membership[i].password) <<
" "
        << membership[i].isTersedia << "\n";
    }
    file.close();
}

void loadDatabaseAdmin() {
    ifstream file(File_Admin);
    if (!file.is_open()) return;
    string line;
    jumlah_admin = 0;
    while (getline(file, line)) {
        if (line.empty()) continue;
        stringstream ss(line);
        string temp;

```

```

        admin admin_temp;

        getline(ss, admin_temp.username, ' ');
        getline(ss, admin_temp.password);

        admin_temp.username =
replaceUnderscoretoSpace(admin_temp.username);
        admin_temp.password =
replaceUnderscoretoSpace(admin_temp.password);
        staff[jumlah_admin] = admin_temp;
        jumlah_admin++;
    }
    file.close();
}

void sortBukuBestSeller() {
    buku* daftarPointer[MAX_RAK * MAX_SLOT]; int jml = 0;
    for (int i = 0; i < MAX_RAK * MAX_SLOT; i++)
        if (rakBuku[i].isTersedia) daftarPointer[jml++] =
&rakBuku[i];

    if (jml == 0) { cout << "Belum ada data buku!\n"; aksiSelesai();
return; }

    for (int i = 0; i < jml - 1; i++)
        for (int j = 0; j < jml - i - 1; j++)
            if (daftarPointer[j]->jumlahBest < daftarPointer[j+1]-
>jumlahBest) {
                buku* temp = daftarPointer[j];
                daftarPointer[j] = daftarPointer[j+1];
                daftarPointer[j+1] = temp;
            }

    cout << "=== DAFTAR BUKU BEST SELLER ===\n";
    for (int i = 0; i < jml; i++)

```

```

        cout << "Judul: " << daftarPointer[i]->namaBuku << "\n"
            << "Penulis: " << daftarPointer[i]->namaPenulis << "\n"
            << "Tahun Terbit: " << daftarPointer[i]->tahunTerbit <<
"\n"

            << "Kategori: " << daftarPointer[i]->kategori << "\n"
            << "Harga: " << daftarPointer[i]->harga << "\n"
            << "Terjual: " << daftarPointer[i]->jumlahBest << "
pcs\n"

            << "Sisa: " << daftarPointer[i]->jumlahStok << "\n-----
-----\n";
    }

void daftarbuku() {
    cout << "=== DAFTAR BUKU ===\n\n"; bool ada = false;
    for (int i = 0; i < MAX_RAK * MAX_SLOT; i++) {
        int nomorRak = (i / MAX_SLOT) + 1;
        int nomorSlot = (i % MAX_SLOT) + 1;
        if (rakBuku[i].isTersedia) {
            cout << " -> [Rak " << nomorRak << ", Slot " <<
nomorSlot << "]" << "\n"
                << "Nama Buku: " << rakBuku[i].namaBuku << "\n"
                << "Penulis: " << rakBuku[i].namaPenulis << "\n"
                << "Tahun Terbit: " << rakBuku[i].tahunTerbit <<
"\n"

                << "Kategori: " << rakBuku[i].kategori << "\n"
                << "Harga: " << rakBuku[i].harga << "\n"
                << "Stok: " << rakBuku[i].jumlahStok << " pcs\n\n";
            ada = true;
        }
    }
    if (!ada) cout << "[ Seluruh Rak Buku Masih Kosong ]\n";
}

void cariBukuJudul() {
    string keyword;

```

```

cout << "=== CARI JUDUL BUKU ===\n"
    << "Masukkan Judul: ";
getline(cin, keyword);

bool f = false;
for (int i = 0; i < MAX_RAK * MAX_SLOT; i++) {
    buku* ptrBuku = &rakBuku[i];
    int nomorRak = (i / MAX_SLOT) + 1;
    int nomorSlot = (i % MAX_SLOT) + 1;
    if ((ptrBuku->isTersedia) && ptrBuku->namaBuku == keyword) {
        cout << " -> [Rak " << nomorRak << ", Slot " <<
nomorSlot << "]\n"
            << "Nama Buku: " << ptrBuku->namaBuku << "\n"
            << "Penulis: " << ptrBuku->namaPenulis << "\n"
            << "Tahun Terbit: " << ptrBuku->tahunTerbit << "\n"
            << "Kategori: " << ptrBuku->kategori << "\n"
            << "Harga: " << ptrBuku->harga << "\n"
            << "Stok: " << ptrBuku->jumlahStok << " pcs\n";
        f = true;
    }
}
if (!f) cout << "Buku \"" << keyword << "\" tidak ditemukan.\n";
}

void beliBuku(float diskon) {
    string l, k, j;
    int r, s, jmlBeli;
    pelanggan p;
    bool koordinatValid = false;
    do {
        cout << "=== BELI BUKU ===\n"
            << "Masukkan No Rak & Slot (misal: 1 2): ";
        cin >> l >> k;
        cin.ignore(1000, '\n');
    }
}

```

```

        if(!cek_input(l, r) || !cek_input(k, s)) {
            cout << "Input rak dan slot harus berupa angka
valid!\n\n";
            r = 0; s = 0;
            continue;
        }
        if (r >=1 && r < MAX_RAK && s >= 1 && s < MAX_SLOT) {
            koordinatValid = true;
        } else {
            cout << "Nomor rak atau slot di luar batas jangkauan!
Silahkan coba lagi.\n\n";
        }

    } while (!koordinatValid);
    int indeks = ((r - 1) * MAX_SLOT) + (s - 1);
    if (rakBuku[indeks].isTersedia && rakBuku[indeks].jumlahStok >
0) {
        cout << "\nBuku Terpilih:\n"
            << "Nama Buku      : " << rakBuku[indeks].namaBuku <<
"\n"
            << "Harga (perbuku): " << rakBuku[indeks].harga -
(rakBuku[indeks].harga * diskon) << "\n"
            << "Stok Tersedia  : " << rakBuku[indeks].jumlahStok <<
" pcs\n";
        do {
            cout << "Jumlah Beli: "; cin >> j;
            cin.ignore(1000, '\n');

            if (!cek_input(j, jmlBeli)) {
                cout << "Input jumlah harus berupa angka
bulat!\n\n";
                jmlBeli = 0;
                continue;
            }
            if (jmlBeli <= 0 || jmlBeli >

```

```

rakBuku[indeks].jumlahStok) {
    cout << "Jumlah tidak valid atau stok buku tidak
mencukupi!\n\n";
    jmlBeli = 0;
}
} while (jmlBeli <= 0);

cout << "Nama Anda: "; getline(cin, p.namaPelanggan);
cout << "Tanggal (DD-MM-YYYY): "; getline(cin, p.tanggal);
rakBuku[indeks].jumlahStok -= jmlBeli;
if (rakBuku[indeks].jumlahStok == 0){
    rakBuku[indeks].isTersedia = false;
}
rakBuku[indeks].jumlahBest += jmlBeli;
p.namaBukuDibeli = rakBuku[indeks].namaBuku;
p.jumlahBeli = jmlBeli;

saveDatabaseBuku();
saveTransaksi(p);
cout << "\nPembelian Berhasil!\n";
} else {
    cout << "Maaf, buku di lokasi Rak " << r << " Slot " << s <<
" sedang kosong atau tidak tersedia!\n";
}
}

void kategoribuku() {
    string pil;
    int pilAngka;
    do{
        cout << "=== KATEGORI BUKU ===\n"
            << "1. Fiksi\n"
            << "2. Non-Fiksi\n"
            << "3. Kembali\n"
            << "Pilih: "; cin >> pil;
    } while (pil != "1" && pil != "2" && pil != "3");
}

```



```

cin.ignore(1000, '\n');

if(!cek_input(pil, pilAngka)) {
    pilAngka = 0;
    cout << "Input harus berupa angka valid\n";
    aksiSelesai(); continue;
}
if (pilAngka == 3) {
    continue;
}
if (pilAngka < 1 || pilAngka > 2) {
    cout << "Pilihan menu tidak tersedia!\n";
    aksiSelesai();
    continue;
}
string katTarget = (pilAngka == 1) ? "Fiksi" : "Non-Fiksi";
string katTargetLower = (pilAngka == 1) ? "fiksi" : "non-
fiksi";
cout << "=== " << katTarget << " ===\n";
bool ketemu = false;
for (int i = 0; i < MAX_RAK * MAX_SLOT; i++) {
    int nomorRak = (i / MAX_SLOT) + 1;
    int nomorSlot = (i % MAX_SLOT) + 1;
    if (rakBuku[i].isTersedia && (rakBuku[i].kategori ==
katTarget || rakBuku[i].kategori == katTargetLower)) {
        cout << " -> [Rak " << nomorRak << "][Slot " <<
nomorSlot << "]" << "\n"
            << "Nama Buku      : " << rakBuku[i].namaBuku <<
"\n"
            << "Harga (Diskon): " << rakBuku[i].harga <<
"\n"
            << "Stok Tersedia: " << rakBuku[i].jumlahStok
<< " pcs\n-----\n";
        ketemu = true;
    }
}

```

```

    }
    if (!ketemu) cout << "Kategori ini belum tersedia.\n";
    aksiSelesai();
}while(pilAngka != 3);
}

void tambahbuku(int rak, int slot) {
    string temp, kat;
    int sub;
    int indeks = -1;
    for (int i = 0; i < MAX_RAK * MAX_SLOT; i++) {
        if (!rakBuku[i].isTersedia) {
            indeks = i;
            break;
        }
    }
    if (indeks == -1) {
        cout << "Maaf, seluruh rak dan slot buku sudah penuh!\n";
        return;
    }
    int nomorRak = (indeks / MAX_SLOT) + 1;
    int nomorSlot = (indeks % MAX_SLOT) + 1;

    cout << "=== TAMBAH BUKU (Otomatis) ===\n";
    cout << "Lokasi Buku: Rak " << nomorRak << ", Slot " <<
nomorSlot << "\n";
    cout << "-----\n";
    string buku;
    bool sudahAda;
    do{
        sudahAda = false;
        cout << "Judul: "; getline(cin, buku);
        for (int i = 0; i < (MAX_RAK * MAX_SLOT); i++) {
            if (buku == rakBuku[i].namaBuku){
                sudahAda = true;

```

```

        break;
    }
}
}while (sudahAda);
rakBuku[indeks].namaBuku = buku;
cout << "Penulis: "; getline(cin, rakBuku[indeks].namaPenulis);

do{
    cout << "Tahun Terbit: "; cin >> temp;
    cin.ignore(1000, '\n');
    if(!cek_input(temp, sub)) {
        sub = 0;
        cout << "Input harus berupa angka valid\n";
    }
}while(sub == 0);
rakBuku[indeks].tahunTerbit = sub;

do {
    cout << "Kategori (Fiksi/Non-Fiksi): "; getline(cin, kat);
}while(kat != "fiksi" && kat != "Fiksi" && kat != "non-fiksi" &&
kat != "Non-Fiksi");

rakBuku[indeks].kategori = kat;
do{
    sub = 0;
    cout << "Harga: "; cin >> temp;
    cin.ignore(1000, '\n');
    if(!cek_input(temp, sub)) {
        sub = 0;
        cout << "Input harus berupa angka valid\n";
    }
}while(sub == 0);
rakBuku[indeks].harga = sub;

do{

```

```

        sub = 0;
        cout << "Stok: "; cin >> temp;
        cin.ignore(1000, '\n');
        if(!cek_input(temp, sub)) {
            sub = 0;
            cout << "Input harus berupa angka valid\n";
        }
    }while(sub == 0);
    rakBuku[indeks].jumlahStok = sub;

    rakBuku[indeks].jumlahBest = 0;
    rakBuku[indeks].isTersedia = true;
    saveDatabaseBuku();
    cout << "\nBuku berhasil didaftarkan!\n";
}

void tambahstok(int rak, int slot) {
    string tam;
    int tamAngka;
    int indeks = ((rak - 1) * MAX_SLOT) + slot - 1;
    cout << "Nama Buku: " << rakBuku[indeks].namaBuku << "\n";
    cout << "Stok lama: " << rakBuku[indeks].jumlahStok << "\n";
    do{
        cout << "Tambahan Stok: "; cin >> tam;
        cin.ignore(1000, '\n');

        if(!cek_input(tam, tamAngka)) {
            tamAngka = 0;
            cout << "Jumlah tidak valid atau stok kurang\n";
        }
    }while(tamAngka <= 0);
    rakBuku[indeks].jumlahStok += tamAngka;
    saveDatabaseBuku();
    cout << "Stok diperbarui!\n";
}

```

```

}

void hapusBuku(int ind) {
    for (int j = ind; j < (MAX_RAK * MAX_SLOT) - 1; j++) {
        if (rakBuku[j].isTersedia) {
            rakBuku[j] = rakBuku[j + 1];
        }
    }
    saveDatabaseBuku();
    cout << "sudah dihapus.\n";
}

void kelolaMembership() {
    string pil;
    int pilAngka;
    do {
        loadDatabaseMember();
        cout << "=== KELOLA MEMBERSHIP ===\n"
             << "1. Daftar Semua Member\n"
             << "2. Nonaktifkan Member\n"
             << "3. Hapus Member Permanen\n"
             << "4. Kembali ke Menu Admin\n"
             << "Pilih: "; getline(cin, pil);
        system("cls");

        if(!cek_input(pil, pilAngka)) {
            pilAngka = 0;
            cout << "Input harus berupa angka valid\n";
            aksiSelesai(); continue;
        }

        if (pilAngka == 1) {
            cout << "=== DAFTAR MEMBER ===\n";
            if (jumlah_member == 0) cout << "[ Belum ada data member
]\n";

```

```

        for (int i = 0; i < jumlah_member; i++) {
            cout << "Username: " << membership[i].username <<
"\n"
            << "Status: " << (membership[i].isTersedia ?
"Aktif" : "Nonaktif") << "\n\n";
            } aksiSelesai(); continue;
        }
        else if (pilAngka == 2) {
            string target;
            cout << "=== NONAKTIFKAN MEMBER ===\n"
            << "Masukkan Username Member: "; getline(cin,
target);

            bool ketemu = false;
            for (int i = 0; i < jumlah_member; i++) {
                if (membership[i].username == target) {
                    if (!membership[i].isTersedia) {
                        cout << "Member memang sudah dalam keadaan
nonaktif.\n";
                    } else {
                        membership[i].isTersedia = false;
                        saveAllMembers();
                        cout << "Member \"" << target << "\"
berhasil dinonaktifkan!\n";
                    }
                    ketemu = true;
                    break;
                }
            }
            if (!ketemu) cout << "Username tidak ditemukan.\n";
            aksiSelesai(); continue;
        }
        else if (pilAngka == 3) {
            string target;
            cout << "=== HAPUS MEMBER PERMANEN ===\n"

```

```

        << "Masukkan Username Member yang akan dihapus: ";
getline(cin, target);

    bool ketemu = false;
    for (int i = 0; i < jumlah_member; i++) {
        if (membership[i].username == target) {
            for (int j = i; j < jumlah_member - 1; j++) {
                membership[j] = membership[j + 1];
            }
            jumlah_member--;
            saveAllMembers();
            cout << "Member \"" << target << "\" berhasil
dihapus secara permanen.\n";
            ketemu = true;
            break;
        }
    }
    if (!ketemu) cout << "Username tidak ditemukan.\n";
}
aksiSelesai();
} while (pilAngka != 4);
}

void halamanAdmin() {
    string pil;
    int pilAngka, r, s;
    do{
        cout << "=== MENU ADMIN ===\n"
            << "1. Daftar Semua Buku\n"
            << "2. Tambah Buku\n"
            << "3. Tambah Stok\n"
            << "4. Hapus Buku\n"
            << "5. Riwayat Transaksi\n"
            << "6. Kelola Member\n"
            << "7. Keluar\n"

```

```

        << "Pilih: "; getline(cin, pil);
system("cls");

if(!cek_input(pil, pilAngka)) {
    pilAngka = 0;
    cout << "Input harus berupa angka valid\n";
    aksiSelesai();
    continue;
}
if (pilAngka < 1 || pilAngka > 7) {
    cout << "Pilihan menu tidak tersedia!\n";
    aksiSelesai();
    continue;
}
if (pilAngka == 1) {
    daftarbuku();
} else if (pilAngka == 5) {
    lihatRiwayatPembelian(); aksiSelesai();
} else if (pilAngka == 6) {
    kelolaMembership();
    continue;
}

if (pilAngka >= 3 && pilAngka <= 4) {
    string l, k;
    cout << "Masukkan No Rak & Slot (misal 1 2): "; cin >> l
>> k;

    cin.ignore(1000, '\n');
    if(!cek_input(l, r)) r = 0;
    if(!cek_input(k, s)) s = 0;
    if (r == 0 || s == 0){
        cout << "Input rak atau slot harus berupa angka
valid\n";

        aksiSelesai(); continue;
    }
}

```



```

        if (r <= 0 || r > MAX_RAK || s <= 0 || s > MAX_SLOT) {
cout << "Koordinat di luar batas!\n"; aksiSelesai(); continue; }
    }
    int indeks = ((r - 1) * MAX_SLOT) + (s - 1);
    if (pilAngka == 2) {
        tambahbuku(r, s);
    } else if (pilAngka == 3 && rakBuku[indeks].isTersedia) {
        tambahstok(r,s);
    } else if (pilAngka == 4 && rakBuku[indeks].isTersedia) {
        cout << "Nama Buku: " << rakBuku[indeks].namaBuku <<
"\n";
        hapusBuku(indeks);
    }
    aksiSelesai();
}while(pilAngka != 7);
}

void halamanUser(float d) {
    string pil;
    int pilAngka;
    do{
        cout << "\n=== TOKO BUKU SINAR ABADI ===\n"
            << "1. Daftar Semua Buku\n"
            << "2. Cari via Kategori\n"
            << "3. Cari Judul\n"
            << "4. Best Seller\n"
            << "5. Beli Buku\n"
            << "6. Keluar\n"
            << "Pilih: "; getline(cin,pil);
        system("cls");

        if(!cek_input(pil, pilAngka)) {
            pilAngka = 0;
            cout << "Input harus berupa angka valid\n";
            aksiSelesai();

```

```

        continue;
    }
    if (pilAngka < 1 || pilAngka > 6) {
        cout << "Pilihan menu tidak tersedia!\n";
        aksiSelesai();
        continue;
    }
    if (pilAngka == 1) daftarbuku();
    else if (pilAngka == 2) kategoribuku();
    else if (pilAngka == 3) cariBukuJudul();
    else if (pilAngka == 4) sortBukuBestSeller();
    else if (pilAngka == 5) beliBuku(d);
    aksiSelesai();
}while(pilAngka != 6);
cin.clear();
}

bool buatMember() {
    member member_temp;
    cout << "====membuat member====\n";
    loadDatabaseMember();
    bool sudahAda;
    do{
        sudahAda = false;
        cout << "silahkan masukkan\n";
        cout << "Username: "; getline(cin, member_temp.username);
        for (int i = 0; i < jumlah_member; i++) {
            if (member_temp.username == membership[i].username) {
                sudahAda = true;
                break;
            }
        }
    }
    if (sudahAda) {
        cout << "\nUsername sudah tersedia / digunakan orang
lain!\n";
    }
}

```

```

    }
    }while(sudahAda);

    cout << "Password: "; getline(cin, member_temp.password);
    member_temp.isTersedia = true;
    if (!member_temp.username.empty() &&
!member_temp.password.empty()) {
        saveDatabaseMember(member_temp);
        return true;
    }else {
        cout << "anda belum terdaftar sebagai membership\n";
        return false;
    }
}

bool prosesLogin(string type, int maxAttempt) {
    string u, p;

    if (maxAttempt == 3) {
        (type == "A" ? loadDatabaseAdmin() : loadDatabaseMember());
    }
    if (maxAttempt <= 0) {
        cout << "Login Gagal! Kesempatan Anda telah habis.\n";
        aksiSelesai();
        return false;
    }

    cout << "=== LOGIN " << (type == "A" ? "ADMIN" : "MEMBER") << "
===\n"

        << "Username: "; getline(cin, u);
    cout << "Password: "; getline(cin, p);

    if (type == "A") {
        for (int i = 0; i < jumlah_admin; i++) {
            if (u == staff[i].username && p == staff[i].password) {

```

```

        cout << "Login Berhasil sebagai Admin!\n";
        aksiSelesai();
        halamanAdmin();
        return true;
    }
}
else {
    for (int i = 0; i < jumlah_member; i++) {
        if (u == membership[i].username && p ==
membership[i].password) {
            if (membership[i].isTersedia) {
                cout << "Login Berhasil sebagai Member!\n";
                cout << "Anda mendapatkan diskon 5%\n";
                aksiSelesai();
                float diskon = 0.05;
                halamanUser(diskon);
                return true;
            } else {
                maxAttempt--;
                cout << "Login gagal!\n";
                cout << "Karena Anda bukan lagi membership\n";
                aksiSelesai();
                return prosesLogin(type, maxAttempt);
            }
        }
    }
}
maxAttempt--;
cout << "Salah! Sisa percobaan: " << maxAttempt << "\n";
aksiSelesai();
cout << "\n";
return prosesLogin(type, maxAttempt);
}

```

```

int main() {
    loadDatabaseBuku();
    string pil;
    int pilAngka;
    do{
        cout << "=== MENU UTAMA ===\n"
            << "1. Pelanggan / Member\n"
            << "2. Petugas Admin\n"
            << "3. Keluar\n"
            << "Pilih: "; getline(cin, pil);
        system("cls");

        if(!cek_input(pil, pilAngka)){
            pilAngka = 0;
            cout << "Input harus berupa angka valid\n";
            aksiSelesai();
            continue;
        }
        if (pilAngka < 1 || pilAngka > 3) {
            cout << "Pilihan menu tidak tersedia!\n";
            aksiSelesai();
            continue;
        }
        if (pilAngka == 3) {
            cout << "Sampai jumpa!\n"; break;
        }
        if (pilAngka == 1) {
            char ans;
            cout << "Masuk akun membership (y/n)? "; cin >> ans;
            cin.ignore(1000, '\n');
            system("cls");

            if (ans == 'y' || ans == 'Y') {
                prosesLogin("U", 3);
            }
        }
    }
}

```

```

        else {
            char create;
            cout << "Mau membuat akun membership(y/n)? "; cin >>
create;

            cin.ignore(1000, '\n');
            system("cls");
            if (create == 'y' || create == 'Y') {
                if (!buatMember()) {
                    aksiSelesai();
                    main();
                }else {
                    cout << "Anda mendapatkan diskon 5%\n";
                    aksiSelesai();
                    float diskon = 0.05;
                    halamanUser(diskon);
                }
            }else {
                float diskon = 0;
                halamanUser(diskon);
            }
        }
    } else if (pilAngka == 2) {
        prosesLogin("A", 3);
    }
}while(pilAngka != 3);
return 0;
}

```

BAB IV

JADWAL Pengerjaan TUGAS DAN PEMBAGIAN TUGAS

4.1. Jadwal Pengerjaan

Tabel 4.1 Tabel Jadwal Pengerjaan

No.	Kegiatan	Mei/Juni			
		1	2	3	4
1.	Mengkaji Masalah		Mei		
2.	Perancangan Sistem Program		Mei		
3.	Pengodean Program			Mei	
4.	Uji Coba dan <i>Debugging</i>			Mei	Mei
5.	Revisi Kode	Juni			
6.	Penyusunan Laporan	Juni	Juni		

4.2. Pembagian Tugas

Kami membagi tugas sesuai dengan tanggung jawab masing-masing setiap kegiatan.

Tabel 4.2 Tabel Pembagian Tugas

No.	Kegiatan	Penanggung Jawab
1.	Mengkaji Masalah	Azriel Aufa Risq/123250120 Muhammad Dzakwan Sundoro Adi/123250121
2.	Perancangan Sistem Program	Azriel Aufa Risq/123250120
3.	Pengodean Program	Muhammad Dzakwan Sundoro Adi/123250121
4.	Uji Coba dan <i>Debugging</i>	Azriel Aufa Risq/123250120 Muhammad Dzakwan Sundoro Adi/123250121
5.	Revisi Kode	Muhammad Dzakwan Sundoro Adi/123250121
6.	Penyusunan Laporan	Azriel Aufa Risq/123250120

		Muhammad Dzakwan Sundoro Adi/123250121
--	--	---

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Dari pembuatan proyek akhir ini, dapat disimpulkan bahwa implementasi konsep pemrograman menggunakan gabungan *struct* dan manipulasi *array* sudah sangat mumpuni untuk membangun sistem manajemen logika yang cukup kompleks. Penggunaan *fstream* terbukti efektif untuk menciptakan basis data (*txt file*) tanpa harus bergantung pada sistem di luar C++. Optimasi dengan algoritma *Bubble Sort* berbasis *pointer* dan sistem *error handling* pada *input* juga menjaga keamanan memori program agar tidak mudah memicu sistem *crash* (seperti program keluar sendiri).

5.2 Saran

Program ini masih memiliki beberapa batasan karena berbasis antarmuka teks (CLI). Untuk pengembangan ke depannya, aplikasi dapat ditingkatkan dengan membangun *Graphical User Interface* (GUI) yang lebih ramah pengguna. Selain itu, transisi basis data dari mode manipulasi teks (*txt file*) menjadi pemanggilan basis data relasional (SQL) dapat dipertimbangkan apabila skala program dan kapasitas data inventaris membesar.

DAFTAR PUSTAKA

<https://github.com/dwzIo/Praktikum-Alpro-Project.git>